B+ Tree Indexing of 10 Million Records: Memory-only, Disk-only and Memory+Disk

Name: Tushar Kant · **Roll No:** B23ES1033 · Oct 10, 2026

1. Introduction and objective

This project implements a B+ Tree index from scratch in C++17, without any index or database library, and compares three storage strategies on a dataset of 10,000,000 randomly shuffled records (bplus_tree_data_10m.csv). All three strategies share the same tree code, so differences in the results come only from where the pages are stored.

The assignment asks for three implementations:

- **Memory-only B+ Tree:** every node lives in RAM.
- **Disk-only B+ Tree:** every node lives in a file and every page access goes to the file, with no cache.
- **Memory + Disk B+ Tree:** nodes live in a file, with a fixed-size LRU buffer pool in RAM.

For each implementation the report analyses insertion time, search time, memory usage, disk usage, tree height and disk I/O, and states the assumptions on page size, tree order and storage structure.

Headline result (10M records): all 10,000,000 records were indexed and verified in every mode. The index is 1562.8 MB with height 4. Memory-only inserts at 0.90 µs per record and searches at 0.66 µs on average, but needs about 1.6 GB of RAM (1580.1 MB peak). Disk-only needs only 19.4 MB of RAM, but inserting took 303.6 s (30.4 µs per record) and a cold search averaged 44.75 µs. The buffered tree cuts page reads per insert from 3.89 (disk-only) to between 1.06 and 0.54, depending on pool size.

2. Dataset and assumptions

The index key is a unique alphanumeric string and the payload is random data. All design parameters below are fixed assumptions of the implementation and are chosen so that one tree node equals one disk page.

Parameter	Value	Reason
Page / node size	4096 bytes	Matches the typical 4 KB NTFS cluster and the 4 KB x86/x64 memory page, so one node is one aligned I/O
Key	12-byte fixed slot, zero-padded, compared with memcmp	The longest key in the dataset fits in 12 bytes; the program aborts rather than truncate a longer key
Payload	100-byte fixed slot, zero-padded	The longest payload fits in 100 bytes; the program aborts rather than truncate a longer payload
Record size in a leaf	112 bytes (12 + 100)	Fixed-width records keep node layout simple
Node header	8 bytes (is_leaf, num_keys, next_leaf)	Shared by leaf and internal nodes
Internal node capacity	255 keys, 256 children	(4096 − 8 − 4) / (12 + 4) = 255.25, rounded down to 255
B+ Tree order (fan-out)	256	Maximum children of an internal node
Leaf node capacity	36 records of 112 bytes	(4096 − 8) / 112 = 36.5, rounded down to 36
Leaf links	next_leaf page number	Leaves form a left-to-right chain for range scans
Metadata	Page 0	Root page, height, record count, leaf and internal counts, key/payload/page sizes
Data pages	Page 1 onwards	Allocated sequentially; byte offset = page number × 4096
Duplicate keys	Existing key is overwritten	The benchmark reported 0 duplicate keys among the 10,000,000 inserts, so keys are unique

Parameter	Value	Reason
Deletes	Not implemented	Not required by the assignment

Dataset. The file bplus_tree_data_10m.csv holds 10,000,000 records in key,payload form in random order. The program is built with 12-byte key slots and 100-byte payload slots (compile options KEY_LEN=12 and PAYLOAD_LEN=100). The reader aborts with an error if any row has a longer key or payload, and it did not abort on this file, so nothing was truncated. Shorter keys and payloads are zero-padded. In fixed slots the raw data is 10,000,000 × 112 bytes = 1,120,000,000 bytes (1068.1 MB).

3. B+ Tree design

The tree is page-based: nodes refer to each other by page number, never by pointer, which lets the same code run on RAM or on a file. It talks to storage only through three calls: get(page) returns a page, mark_dirty(page) records a modification, and alloc() returns a new zeroed page.

Node layout. Every page is exactly 4096 bytes. An internal node stores 256 child page numbers followed by 255 separator keys. A leaf stores 36 keys followed by 36 payloads, plus the page number of its right sibling. Keys are compared as raw bytes, which gives lexicographic order.

Search. Starting at the root, each internal node is binary-searched for the child whose key range contains the search key. After height − 1 steps the leaf is binary-searched for an exact match. A search therefore reads exactly one page per level, which is 4 pages for 10M records.

Insert. Insertion is a recursive top-down descent to the correct leaf.

1. If the leaf has room, the record is inserted in sorted position.
2. If the leaf is full, its 36 records plus the new one (37) are split into 18 and 19 records, and the first key of the right leaf is copied up to the parent.
3. If the parent is also full, it is split the same way, but the middle key moves up instead of being copied. With 256 keys after the insert, the left node keeps 128 keys, the middle key moves up, and the right node gets 127 keys.
4. If the root splits, a new root with one key and two children is created and the height grows by one.
5. If the key already exists, its payload is overwritten and the record count is unchanged.

Leaf chain. Splitting a leaf links the new page into the chain, so a range query descends once and then follows next_leaf pointers. A range_count function using this chain is included and tested.

Frame-safety rule. A page pointer returned by get() is never used after another get() or alloc(), because a store may recycle that memory. The code keeps page numbers across calls and re-fetches pages, which removes a class of stale-pointer bugs in the disk-backed stores.

Expected size. Random inserts into a B+ Tree leave leaves about 69% full on average (ln 2). For 10M records this predicts about 10,000,000 / (36 × 0.69) ≈ 402,600 leaves. The measured value is 398,023 leaves (69.8% fill) plus 2,057 internal nodes.

4. Three storage back-ends

The tree class is a C++ template over a Store type, so the three implementations differ only in the store. Each store reports its own I/O counters.

Property	Memory-only (MemStore)	Disk-only (DiskStore)	Memory + Disk (BufferedStore)
Where pages live	RAM arena in 64 MB chunks	Index file only	Index file plus RAM buffer pool
Page cache	Not applicable, all pages resident	None: 2 scratch pages	LRU pool of N pages (default 4096 = 16 MB)
Page read	Pointer arithmetic	One pread per page not in scratch	Pool hit, or one pread on a miss
Page write	None	Write-through: one pwrite per modified page	Write-back: only on eviction or final flush
Lookup structure	Chunk index	Two-slot scan	Open-addressing hash table, page number to frame
New page	Zeroed memory	Zeroed scratch page, marked dirty	Zeroed frame in the pool, marked dirty
Persistent	No	Yes	Yes

Memory-only. Pages are allocated in 64 MB chunks that never move, so pointers stay valid and no large copy happens as the tree grows.

Disk-only. The two scratch slots exist only so the code can look at one page and the page it just touched. Every other access is a real file read, and every modification is written immediately. This models a tree with no buffer manager.

Memory + Disk. Frames form an intrusive doubly linked LRU list. A hash table maps page numbers to frames. When the pool is full, the least recently used frame is evicted, and written to the file only if it was modified. Pool sizes of 16 MB, 64 MB and 256 MB (4096, 16384 and 65536 pages) were benchmarked.

File access. The reported runs used the Windows code path: one wrapper issues ReadFile and WriteFile with an explicit file offset (pread and pwrite on Linux), checks every return value and stops on an I/O error. At the end of insertion all dirty pages are written and the file is flushed to disk. The cold option empties the program's own buffer pool before searching. On Linux it also asks the kernel to drop its cached copy of the index file; on Windows it cannot, so the operating system's file cache may still serve some page reads.

5. Development process and decisions

The work followed five steps: a first implementation, a code review, a rewrite that fixed the review findings, a test suite, and full-scale benchmarking.

Key design decisions

- **One tree, three stores.** A template parameter keeps the algorithm identical across modes, so the comparison is fair and a bug is fixed once.
- **Page numbers instead of pointers.** This is what makes the same nodes storable in RAM or on disk.
- **Fixed-width keys and payloads.** They give a constant leaf and internal capacity, and make node arithmetic exact.
- **Header kept in RAM, written to page 0.** The 40-byte header is cached in the tree object and written when the root changes and once at the end, instead of rewriting page 0 on every insert, which would have inflated disk-only write counts.
- **No double writes for new pages.** A new page is created zeroed in memory and marked dirty, and reaches the file once, when it is written or evicted.
- **Write-through for disk-only, write-back for buffered.** This is the defining difference between the two disk-backed modes.

- **Fail loudly, never truncate.** Keys or payloads longer than the compiled widths abort the run with the exact rebuild command.

Problems found in the first version and how they were fixed

Problem	Effect	Fix
Keys and payloads silently truncated to 8 and 64 bytes	Longer keys collide, so records are lost and sizes are understated	Reader aborts on over-long keys and payloads; widths configurable at build time
Record count increased on duplicate overwrite	Count could disagree with the leaf chain	insert() returns whether a new key was added
Stale metadata pointer after a root split in disk mode	About a 1-in-16 chance of corrupting the new root on each root split	Header held in the tree object; the old root is saved before any page fetch
Dead, out-of-bounds code in the internal split	Read past the end of the key array	Removed; split rewritten with memmove and a temp area
CSV reader mishandled CRLF at buffer boundaries and stopped on blank lines	Corrupt record or early stop	Line-based reader handling LF, CRLF, blank lines and an optional header
Missing-key test altered the last character	The altered key might exist, giving false errors	Absent keys contain a control byte that cannot be in the data
Node counts skipped for large disk runs	Leaf and internal counts reported as 0	Counters maintained during insert
Page 0 rewritten every insert, new pages written twice	Inflated write counts	Header cached; new pages written once
Insert time included CSV parsing	Timing mixed I/O of the input file with index work	Per-insert timing; parsing time reported separately
Search check did not compare payloads	Wrong payloads could pass	Sample payloads stored and compared

6. Experimental methodology and metrics

Test machine. HP Victus laptop with 8 GB RAM and an SSD, running Windows. The program was compiled in MSYS2 MINGW64 with g++ at -O2 (C++17), with KEY_LEN=12 and PAYLOAD_LEN=100. Absolute times depend on this machine, but the I/O counts and the ratios between modes are properties of the algorithm.

Procedure for each run

1. Stream the CSV once and insert records in file order (the order is random). Each insert is timed individually.
2. Flush all dirty pages and fsync the index file. The flush time is added to the insertion time.
3. Verify the whole tree structure (see section 7).
4. If cold search is requested, empty the buffer pool (and, on Linux only, drop the OS file cache).
5. Reset I/O counters, then run 100,000 searches for keys that exist (every 100th inserted key) and 10,000 searches for keys that cannot exist.
6. Append one row of results to a CSV file.

Metrics

Metric	How it is measured
Insertion time	Sum of per-insert timings plus final flush; CSV parsing excluded. Wall time including parsing is also reported
Search time	Average, median (p50) and 99th percentile over 110,000 queries
Memory usage	Peak resident set size of the process (VmHWM on Linux, PeakWorkingSetSize on Windows)
Disk usage	Number of allocated pages × 4096 bytes
Tree height	Levels from root to leaf, root-only tree = 1
Node counts and leaf fill	Counters maintained by the tree; fill = records / (leaves × 56)

Metric	How it is measured
Disk I/O	Pages and MB read and written, counted at the file-access layer, separately for the insert and search phases
Buffer behaviour	Pool hits, misses and hit ratio, separately for the insert and search phases

Throughput. Throughput is computed as one divided by the average time per operation.

Limitation of the timing data. The page counts are exact. Elapsed times for the disk-backed modes also include effects of the Windows file cache and background write-back, which the program cannot control. This is discussed in section 9.

7. Correctness verification

The tree indexed all 10,000,000 records correctly in every mode. Correctness was checked at three levels.

1. Self-test suite (make test). Each case inserts keys into the tree and into a std::map reference (used only inside the test), then compares them.

- Random, ascending and descending key orders, on all three stores, with up to 200,000 keys.
- A buffered store with only 4 pages, which forces constant eviction and reload.
- Overwriting 10% of existing keys: the insert must report not-new, and the latest payload must be returned.
- Record count equals the number of distinct keys.
- Every key is found with its latest payload, and 2,000 absent keys are not found.
- 200 random range counts equal the reference.
- After dropping all caches, every key is read back correctly from disk.
- Boundary sizes: 1 record, 2 records, exactly one full leaf, one full leaf plus one record (the first split) and three leaves' worth of records.

2. Stress build. The same suite is compiled with 256-byte pages (with 8-byte keys and 64-byte payloads: 3 records per leaf, 20 keys per internal node) under AddressSanitizer and UndefinedBehaviorSanitizer. Tiny pages make the tree 5 levels deep with only 60,000 keys, so internal splits and root growth happen thousands of times. On Linux during development, all tests passed with no memory or undefined-behaviour reports.

3. Checks inside every benchmark run. After inserting, the program walks the entire tree and verifies:

- keys are strictly increasing inside every node;
- every key lies within the range given by its parent's separator keys;
- all leaves are at the same depth;
- the leaf chain is strictly sorted and contains exactly the recorded number of records and leaves.

It then searches 100,000 existing keys, comparing the returned payload with the inserted one, and 10,000 keys that cannot exist.

Outcome for all five 10M configurations: structure check OK with 10,000,000 records in the leaf chain, 100,000 of 100,000 existing keys found with matching payloads, 0 payload mismatches, and 0 of 10,000 false positives.

The structure check proves all 10M records are present, ordered and reachable. The 10M benchmark does not look up every one of the 10M keys individually.

The self-test suite and the sanitizer stress build were run on Linux. The per-run checks in item 3 were produced on the Windows machine for all five 10M configurations, and every one of them passed.

8. Results

All results are for the 10,000,000 records of bplus_tree_data_10m.csv, built with 12-byte keys and 100-byte payloads, on the machine in section 6. Disk-backed searches used the cold option, which empties the buffer pool but does not clear the Windows file cache. Each configuration was run once. The raw rows are in the results.csv file written by the benchmark. MB means 1,048,576 bytes.

8.1 Time and throughput

Configuration	Insert total (s)	Insert avg (µs)	Insert throughput (records/s)	Search avg (µs)	Search p50 (µs)	Search p99 (µs)	Search throughput (queries/s)
Memory-only	9.04	0.90	1,106,700	0.66	0.60	1.40	1,508,300
Disk-only	303.60	30.36	32,900	44.75	16.40	174.20	22,300
Memory + Disk, 16 MB pool	266.58	26.66	37,500	22.46	7.50	150.70	44,500

Configuration	Insert total (s)	Insert avg (µs)	Insert throughput (records/s)	Search avg (µs)	Search p50 (µs)	Search p99 (µs)	Search throughput (queries/s)
Memory + Disk, 64 MB pool	251.63	25.16	39,700	49.35	10.40	202.80	20,300
Memory + Disk, 256 MB pool	314.58	31.46	31,800	63.46	17.60	212.70	15,800

Insert times exclude CSV parsing. Including parsing, wall times were 10.13 s, 305.60 s, 268.41 s, 253.53 s and 316.56 s respectively. Search figures cover 110,000 queries: 100,000 existing keys and 10,000 keys that do not exist.

8.2 Tree structure, memory and disk usage

Configuration	Height	Leaf nodes	Internal nodes	Leaf fill	Index size (MB)	Peak process memory (MB)	Memory held by the store (MB)	Disk usage (MB)
Memory-only	4	398,023	2,057	69.8%	1562.8	1580.1	1562.8	0
Disk-only	4	398,023	2,057	69.8%	1562.8	19.4	0.0	1562.8
Memory + Disk, 16 MB pool	4	398,023	2,057	69.8%	1562.8	35.5	16.1	1562.8
Memory + Disk, 64 MB pool	4	398,023	2,057	69.8%	1562.8	83.9	64.5	1562.8
Memory + Disk, 256 MB pool	4	398,023	2,057	69.8%	1562.8	277.4	258.0	1562.8

The tree has 400,081 pages in total (1,638,731,776 bytes), including the metadata page. The raw data in fixed slots is 1,120,000,000 bytes, so the index is about 1.46 times the raw data.

8.3 Disk I/O during insertion

Configuration	Page reads	Page writes	MB read	MB written	Reads per insert	Writes per insert	Pool hit ratio
Memory-only	0	0	0	0	0	0	not applicable
Disk-only	38,948,599	10,800,156	152,143.0	42,188.1	3.895	1.080	not applicable

Configuration	Page reads	Page writes	MB read	MB written	Reads per insert	Writes per insert	Pool hit ratio
Memory + Disk, 16 MB pool	10,596,473	10,061,346	41,392.5	39,302.1	1.060	1.006	73.35%
Memory + Disk, 64 MB pool	8,292,682	8,692,614	32,393.3	33,955.5	0.829	0.869	79.14%
Memory + Disk, 256 MB pool	5,349,927	5,750,008	20,898.2	22,461.0	0.535	0.575	86.54%

The buffer pool recorded 29,158,600 hits (16 MB), 31,462,391 hits (64 MB) and 34,405,146 hits (256 MB) during insertion; the misses equal the page reads.

8.4 Disk I/O during search (110,000 queries, cold)

Configuration	Page reads	Reads per query	Pool hits	Pool misses	Pool hit ratio
Memory-only	0	0	not applicable	not applicable	not applicable
Disk-only	440,000	4.000	not applicable	not applicable	not applicable
Memory + Disk, 16 MB pool	140,682	1.279	299,318	140,682	68.03%
Memory + Disk, 64 MB pool	108,487	0.986	331,513	108,487	75.34%
Memory + Disk, 256 MB pool	99,922	0.908	340,078	99,922	77.29%

No search wrote any page.

8.5 Insertion progress

Cumulative wall-clock seconds (including CSV parsing) printed by the program after each million records.

Records inserted	Memory-only	Disk-only	Memory + Disk, 16 MB	Memory + Disk, 64 MB	Memory + Disk, 256 MB
1M	0.7	13.3	7.3	3.2	0.8
2M	1.6	28.8	18.4	12.1	3.5
3M	2.5	48.6	32.3	23.3	12.7
4M	3.5	72.6	52.9	40.7	28.9

Records inserted	Memory-only	Disk-only	Memory + Disk, 16 MB	Memory + Disk, 64 MB	Memory + Disk, 256 MB
5M	4.5	99.5	79.2	65.1	62.3
6M	5.6	130.9	108.4	92.1	91.7
7M	6.7	166.3	139.9	121.8	140.1
8M	7.8	203.6	173.8	153.1	190.6
9M	8.9	247.7	217.4	185.2	240.7
10M	10.1	304.4	267.1	252.5	313.2

8.6 Scaling from 1M to 10M records (memory-only)

A trial run on the first 1,000,000 records of the same file, in memory-only mode, shows how cost grows with size.

Records	Height	Leaf nodes	Internal nodes	Index size (MB)	Insert total (s)	Insert avg (µs)	Search avg (µs)	Peak memory (MB)
1,000,000	3	39,769	257	156.4	0.615	0.62	0.47	175.8
10,000,000	4	398,023	2,057	1562.8	9.036	0.90	0.66	1580.1

9. Analysis and discussion

Tree height and size. Ten million records need a tree of height 4. An internal node can hold 256 children, but random insertion leaves nodes partly full, so the average fan-out is lower: about 200 children in the bottom internal level, since 398,023 leaves sit under roughly 2,000 internal nodes. The 2,057 internal nodes occupy about 8 MB, 0.5% of the 1562.8 MB index. The index is about 1.46 times the raw data because leaves are on average 69.8% full, close to the theoretical 69% for random insertion, and because each page carries a header.

Memory-only versus disk-backed. Memory-only is the fastest: 0.90 µs per insert (about 1.1 million inserts per second) and 0.66 µs per search. The price is memory: its peak was 1580.1 MB (about 19% of the machine's 8 GB RAM), about 81 times the 19.4 MB used by disk-only. Disk-only inserted 33.6 times slower (303.6 s against 9.04 s) and its average search was about 67 times slower (44.75 µs against 0.663 µs; the median was 27 times slower). In return it uses almost no RAM and its index survives the process.

Disk-only I/O. Without a cache, a search touches exactly one page per level: 440,000 reads for 110,000 queries, which is 4.000 reads per query and equals the tree height. An insert needed 3.895 reads on average, slightly under 4 because the two scratch pages sometimes already hold the next page. It needed 1.080 writes: one for the modified leaf, plus extra writes on splits. About 0.04 leaf splits happen per insert (398,023 leaves for 10M records), and each split writes the new sibling, the old leaf and the parent instead of one page, which adds about 0.08 writes per insert.

Effect of the buffer pool on I/O. The three pools hold 1.0%, 4.1% and 16.4% of the index. Even the smallest helps a lot, because the internal levels are tiny and every operation uses them.

- **Insert reads per insert:** 3.895 (disk-only) fell to 1.060 (16 MB, 73% fewer), 0.829 (64 MB, 79% fewer) and 0.535 (256 MB, 86% fewer).
- **Insert writes per insert:** 1.080 fell to 1.006, 0.869 and 0.575 (7%, 20% and 47% fewer). Writes fall more slowly because every modified leaf must be written once when it is evicted.
- **Search reads per query:** 4.000 fell to 1.279, 0.986 and 0.908 (68%, 75% and 77% fewer), matching the pool hit ratios in section 8.4.

Returns diminish because the insertion keys are random: each insert lands in an unpredictable one of 398,023 leaves, so the pool holds the hot internal pages but only a fraction of the leaves.

Elapsed times of the disk-backed modes do not follow the I/O counts. Insertion with the 16 MB and 64 MB pools was 12% and 17% faster than disk-only (266.6 s and 251.6 s against 303.6 s). The 256 MB pool did the fewest I/O operations (0.535 reads per insert) but was the slowest, at 314.6 s, 4% slower than disk-only. The progress table in section 8.5 shows how this happened: with the 256 MB pool the first million records took only 0.8 s and the first three million 12.7 s, while the pool was still filling, but millions 7 to 10 took about 48 to 73 s each, slower than disk-only over the same stretch (35 to 57 s each). Disk-only itself slowed from 13.3 s for the first million to 56.7 s for the tenth. With 8 GB of RAM and a 1.6 GB index file, the whole file can fit in the Windows file cache, so a simple cache-size limit is not an obvious explanation. Plausible contributors are the operating system's cache management and write-back, antivirus scanning of the file being written, and the SSD's behaviour under sustained writes, including laptop thermal throttling: these runs wrote between 22 and 41 GB of page data. None of these was measured, so the cause is unconfirmed.

Search latency of the disk-backed modes. Averages were 44.75 µs (disk-only), 22.46 µs (16 MB), 49.35 µs (64 MB) and 63.46 µs (256 MB). In every case the median is far lower than the average (16.4, 7.5, 10.4 and 17.6 µs) and the 99th percentile is 150 to 213 µs, so most queries are served quickly from a cache and a minority wait for the device. The larger pools read fewer pages (0.986 and 0.908 reads per query against 1.279) yet were slower on average. As with insertion, this shows the elapsed time depends on the state of the operating system's cache and the disk, which the program cannot control, and the cold option does not clear the Windows file cache. The page counts are the dependable comparison between the disk-backed modes; the elapsed times are indicative only, since each configuration was run once.

Throughput summary. Memory-only sustains about 1.11 million inserts per second and 1.51 million searches per second. The disk-backed modes sustain about 31,800 to 39,700 inserts per second and 15,800 to 44,500 cold searches per second.

Choosing a mode.

- Memory-only suits an index that fits comfortably in RAM and does not need to persist; at 10M records it needs about 1.6 GB.
- Disk-only suits very small memory budgets (19.4 MB here), but it pays 4 page reads for every search and was the slowest to build apart from the 256 MB run.
- Memory + Disk is the practical choice: memory use is a tunable setting, the index is persistent, and a pool of only 1% of the index already removes 73% of the insert page reads and 68% of the search page reads. The fastest build among the disk-backed runs used the 64 MB pool (251.6 s).

10. Limitations, conclusion and reproduction

10.1 Limitations

- **Single run per configuration.** Each configuration was run once, so no confidence intervals are given, and the elapsed times of the disk-backed modes did not follow the page counts (section 9). The page counts are exact; the timings are indicative.
- **Operating system cache.** The cold option empties the program's buffer pool but cannot clear the Windows file cache, so some disk-backed reads and writes were absorbed by the operating system.
- **Fixed-width slots.** The build uses 12-byte key slots and 100-byte payload slots. Shorter keys and payloads are zero-padded, so the index size reflects 112-byte slots, not the actual text lengths.

- **One machine.** The reported runs come from one Windows laptop. Linux was used only for development and testing.
- **Scope of the index.** No deletion, concurrency control or crash recovery. These were outside the assignment.
- **Sampled lookups.** The benchmark searches 100,000 existing keys, not all 10 million. Presence and ordering of all records is covered by the full structural check.

10.2 Conclusion

A B+ Tree built from scratch with 4096-byte pages, 256-way internal nodes and 36-record leaves indexed all 10,000,000 records in 4 levels and 1562.8 MB, and was verified correct in all three storage modes. Memory-only is fastest (0.90 µs per insert, 0.66 µs per search) but needs about 1.6 GB of RAM. Disk-only needs 19.4 MB of RAM and reads one page per level on every access, which made it about 34 times slower to build and about 67 times slower to search on average. The buffered tree is the tunable middle ground: a pool holding 1% of the index removes 73% of insert page reads, and a pool holding 16% removes 86%.

10.3 Reproduction

The source is a standard C++17 project with no third-party libraries. It needs g++; on Windows, MSYS2 MINGW64 was used. The commands below are the ones used for the reported results.

```
# build with 12-byte keys and 100-byte payloads
g++ -O2 -std=c++17 -DBPT_KEY_LEN=12 -DBPT_PAYLOAD_LEN=100 -o bpt.exe
src/main.cpp -lpsapi

./bpt.exe inspect bplus_tree_data_10m.csv    # key and payload lengths vs the
build
./bpt.exe test                               # self-tests

./bpt.exe mem  bplus_tree_data_10m.csv
./bpt.exe disk bplus_tree_data_10m.csv --cold
./bpt.exe buf  bplus_tree_data_10m.csv --cache-pages 4096  --cold   # 16 MB
pool
./bpt.exe buf  bplus_tree_data_10m.csv --cache-pages 16384 --cold   # 64 MB
pool
./bpt.exe buf  bplus_tree_data_10m.csv --cache-pages 65536 --cold   # 256 MB
pool
```

Each run appends one row to results.csv. On Linux, make builds the program (make KEY_LEN=12 PAYLOAD_LEN=100) and run_all.sh runs everything.

10.4 Repository contents

Path	Purpose
src/bptree.hpp	B+ Tree node layouts, insert, search, range count and verification
src/stores.hpp	MemStore, DiskStore and BufferedStore, plus the portable file wrapper
src/main.cpp	Command line, CSV reader, benchmark, inspect mode and self-tests
tools/gen_dataset.cpp	Optional generator for test data
Makefile, run_all.sh, run_all.bat	Build, test and benchmark scripts
README.md	Build, usage and assumptions
results.csv	Raw measurements behind section 8, written by the benchmark